

Project Title: Vehicle Rental Service

Students: Gulnaz Yeskermes, Bekarys Uzbekbayev, Asylbek Agabekuly

Group: SE-2438

1. Introduction

The goal of this final project is to develop a software system for renting vehicles using at least six design patterns covered in the Software Design Patterns course. The system must demonstrate the ability to apply design patterns in practice, integrate them into a unified architecture, and show how they function within a real-world scenario.

The project “Vehicle Rental Service” allows customers to choose a vehicle, add optional services, complete the rental process, receive notifications, and generate transport-related reports.

To achieve this functionality, the following design patterns were used:

- Factory
- Decorator
- Adapter
- Observer
- Strategy
- Visitor

Each of these patterns is responsible for a specific part of the system’s behavior and is integrated into a single workflow.

2. Body

2.1 Project Description

The vehicle rental system simulates the process of selecting, configuring, and paying for rented vehicles. A customer can choose a vehicle type (car, bike, van), add additional services, select a pricing plan, make a payment, and receive notifications and analytical reports.

This creates a realistic and extendable rental environment.

2.2 Applied Design Patterns

1. Factory

Used to create different types of vehicles without directly instantiating their classes.

This improves extensibility and reduces coupling.

Classes: CarFactory, BikeFactory, VanFactory, VehicleFactoryProvider.

2. Decorator

Allows adding optional services to a rental dynamically, such as GPS or insurance. Each decorator extends the rental's behavior while preserving the original object.

Classes: GpsDecorator, InsuranceDecorator, RentalDecorator.

3. Adapter

Provides integration with an external payment system.

The Adapter allows the rental system to use a third-party API through a unified interface.

Classes: PaymentAdapter, ExternalPaymentService, PaymentProcessor.

4. Observer

Responsible for sending notifications to customers and the administrator whenever a new rental is created.

Classes: RentalSubject, AdminObserver, CustomerObserver.

5. Strategy

Enables selecting different pricing models: hourly, daily, or premium. Each strategy calculates the rental price differently.

Classes: PricingStrategy, HourlyPricingStrategy, DailyPricingStrategy, PremiumPricingStrategy.

6. Visitor

Two visitors were implemented:

- **ReportVisitor** — generates a report for each vehicle

- **DiscountVisitor** — calculates a discount depending on the vehicle type

Visitor allows adding new operations to objects without modifying their classes.

2.3 System Integration of All Patterns

The complete use-case scenario of the system is:

1. The customer selects a vehicle → created via **Factory**
2. The customer adds optional services → applied through **Decorator**
3. The customer selects a pricing plan → calculated using **Strategy**
4. The customer makes a payment → processed via **Adapter**
5. The system sends notifications → handled using **Observer**
6. Reports and discounts are generated → implemented with **Visitor**

Thus, all six patterns work together inside a single coherent system.

3. Conclusion

During the implementation of this final project, a fully functional vehicle rental system was designed using six major software design patterns. Each pattern was implemented appropriately to solve its specific task. All components are successfully integrated in a unified scenario, which is demonstrated by the working execution of the system.

The project proves the ability to apply object-oriented principles and design patterns to build scalable, flexible, and well-structured software solutions.

4. Further Work

The project can be expanded with the following enhancements:

- Implementation of a GUI (JavaFX or Swing)
- Database integration for storing rentals and vehicles
- User authentication and account management
- New pricing strategies and additional vehicle types
- Integration with real-world payment APIs
- Improved reporting tools (PDF/Excel generation)